



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	M. Mario Al Zaky
NPM	2428240157
Kelas	SI5C
Topik Tugas 1	Topik 33: Data Usaha Kecil (UMKM)
Entitas utama	Usaha (Business)
Tautan repositori	https://github.com/Phumax12/si5c_kuis1_Mario
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Topik 33: Data Usaha Kecil (UMKM)
Entitas utama (tunggal)	Usaha (Business)
Alamat dasar	/businesses
Field data	id, namaUsaha, pemilik, kategori, kota, noHp
Field wajib saat POST	namaUsaha, pemilik, kategori, kota (noHp opsional)
Query string untuk filter	kategori, contoh: /businesses?kategori=kerajinan
Data awal	3 data: Anyaman Bambu Sari (kerajinan, Tasikmalaya), Keripik Pisang Bu Ani (kuliner, Bandar Lampung), Batik Tulis Nusantara (fesyen, Pekalongan)
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/businesses	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/businesses/:id	Tidak	200	Ambil satu data
3	POST	/businesses	Ya	201	Tambah data
4	PUT	/businesses/:id	Ya	200	Ubah data
5	DELETE	/businesses/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```

PS C:\si5c_kuis1_Mario> node -v
v24.15.0
PS C:\si5c_kuis1_Mario> npm -v
11.12.1
PS C:\si5c_kuis1_Mario> npm init -y
Wrote to C:\si5c_kuis1_Mario\package.json:

{
  "dependencies": {
    "dotenv": "^18.0.5",
    "express": "^5.2.1"
  },
  "name": "si5c_kuis1_mario",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Phumax12/si5c_kuis1_Mario.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/Phumax12/si5c_kuis1_Mario/issues"
  }
},

Ⓢ PS C:\si5c_kuis1_Mario> nvm install express
FAILED vexpress: unknown alias: express
1 version(s) failed to install
PS C:\si5c_kuis1_Mario> npm install express

added 68 packages in 18s

28 packages are looking for funding
run `npm fund` for details

```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

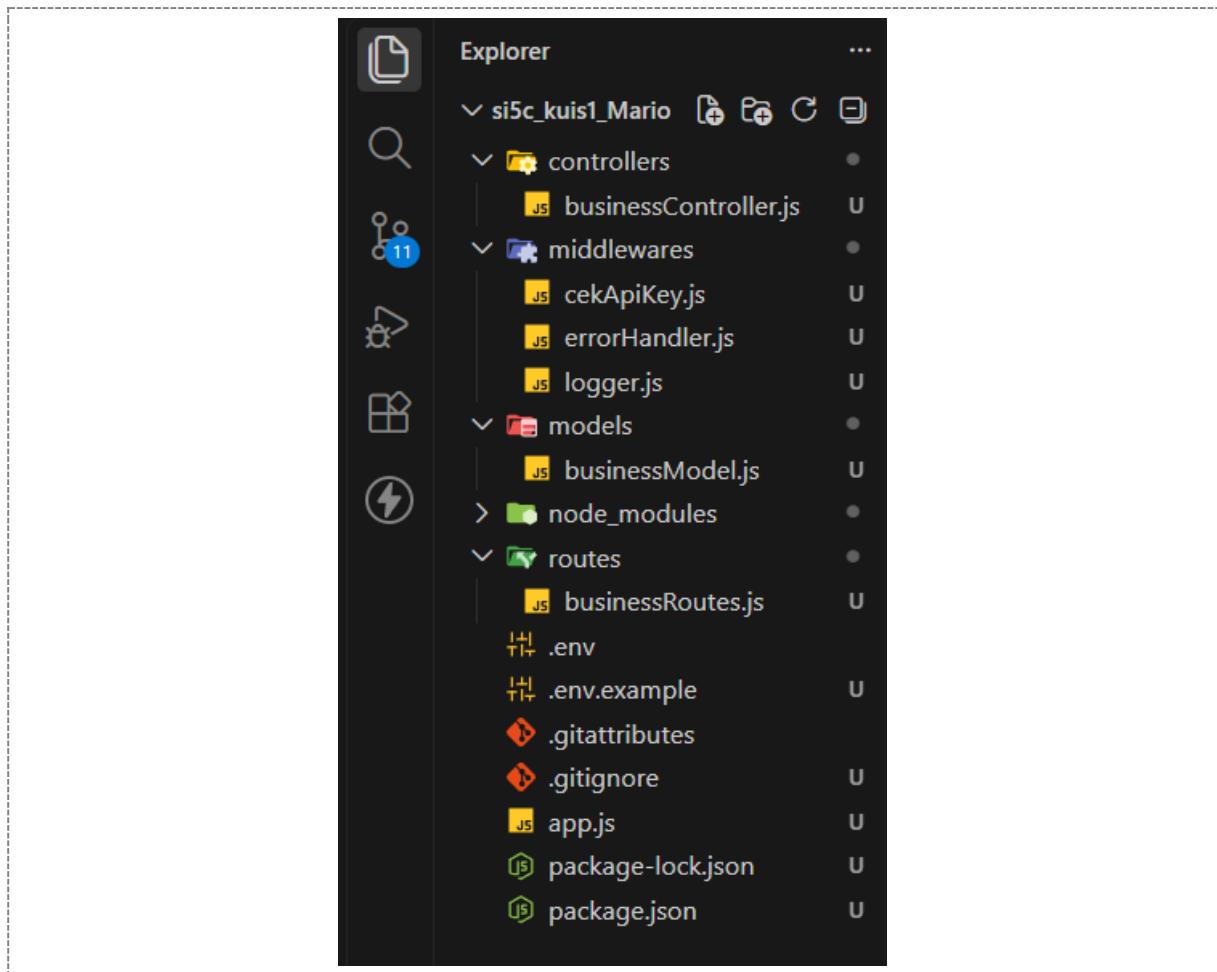
Express adalah framework yang saya pakai untuk membuat server dan mendaftarkan route API usaha (/businesses) tanpa menulis server HTTP dari nol. cors dipasang lewat `app.use(cors())` agar API dapat diakses dari origin lain, misalnya halaman frontend. dotenv membaca file `.env` sehingga PORT dan API_KEY tidak ditulis langsung di kode dan tidak ikut terunggah ke GitHub. Express 5 mensyaratkan Node.js versi 18 atau lebih baru, jadi hasil `node -v` harus memenuhi syarat itu supaya paket terpasang dan server dapat berjalan tanpa error.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

routes/ memetakan alamat URL ke fungsi controller (businessRoutes.js). controllers/ berisi fungsi yang membaca request, memvalidasi input, memanggil model, lalu menyusun respons (businessController.js). models/ menyimpan array businesses dan operasi datanya tanpa mengenal req atau res (businessModel.js). middlewares/ berisi fungsi perantara yang berjalan sebelum atau sesudah controller: logger, cekApiKey, dan errorHandler.

2.2 Ringkasan tugas setiap berkas

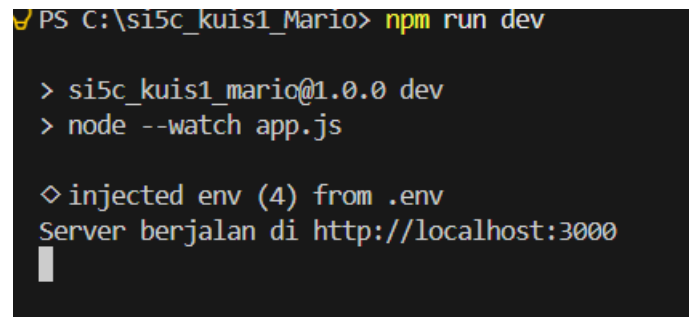
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/businessModel.js	Menyimpan data usaha dalam array businesses beserta penghitung nextId, dan menyediakan fungsi getAll(kategori), getByld(id), create(data), update(id, data), dan remove(id). Tidak ada req atau res di sini, hanya pengelolaan data.
controllers/businessController.js	Berisi getAllBusinesses, getBusinessByld, createBusiness, updateBusiness, dan deleteBusiness. Tiap fungsi membaca req.params, req.query, atau req.body, memvalidasi (namaUsaha, pemilik, kategori, kota wajib pada POST), memanggil businessModel, lalu mengirim respons atau memanggil next(errorHttp(...)).
routes/businessRoutes.js	Membuat express.Router() dan memetakan GET, POST, PUT, DELETE ke fungsi controller. cekApiKey dipasang hanya pada POST, PUT, dan DELETE, sedangkan dua GET dibiarkan terbuka.
middlewares/logger.js	Mencatat waktu, metode, dan alamat setiap request ke terminal, misalnya [waktu] GET /businesses, lalu memanggil next() agar request lanjut.
middlewares/cekApiKey.js	Membandingkan header x-api-key dengan process.env.API_KEY. Jika kosong atau berbeda, memanggil next(errorHttp(401, ...)) sehingga controller tidak dijalankan.
middlewares/errorHandler.js	Berisi helper errorHttp(status, message), notFound untuk rute yang tidak ada, dan errorHandler terpusat yang mengubah semua error, termasuk JSON rusak (entity.parse.failed), menjadi respons JSON yang seragam.
app.js	Titik masuk aplikasi: memuat dotenv, memasang logger, cors, dan express.json(), mendaftarkan app.use("/businesses", businessRoutes), memasang notFound dan errorHandler paling akhir, lalu menjalankan app.listen(PORT).

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
PS C:\si5c_kuis1_Mario> npm run dev

> si5c_kuis1_mario@1.0.0 dev
> node --watch app.js

◇ injected env (4) from .env
Server berjalan di http://localhost:3000
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port berasal dari `process.env.PORT` yang diisi `dotenv` dari file `.env` (di komputer saya 4000); jika tidak ada, kode memakai nilai cadangan 3000 lewat `process.env.PORT || 3000`. Bila port sudah dipakai aplikasi lain, `app.listen` gagal dengan error `EADDRINUSE` dan server tidak berjalan. Solusinya menghentikan aplikasi yang memakai port itu atau mengganti `PORT` di `.env`.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

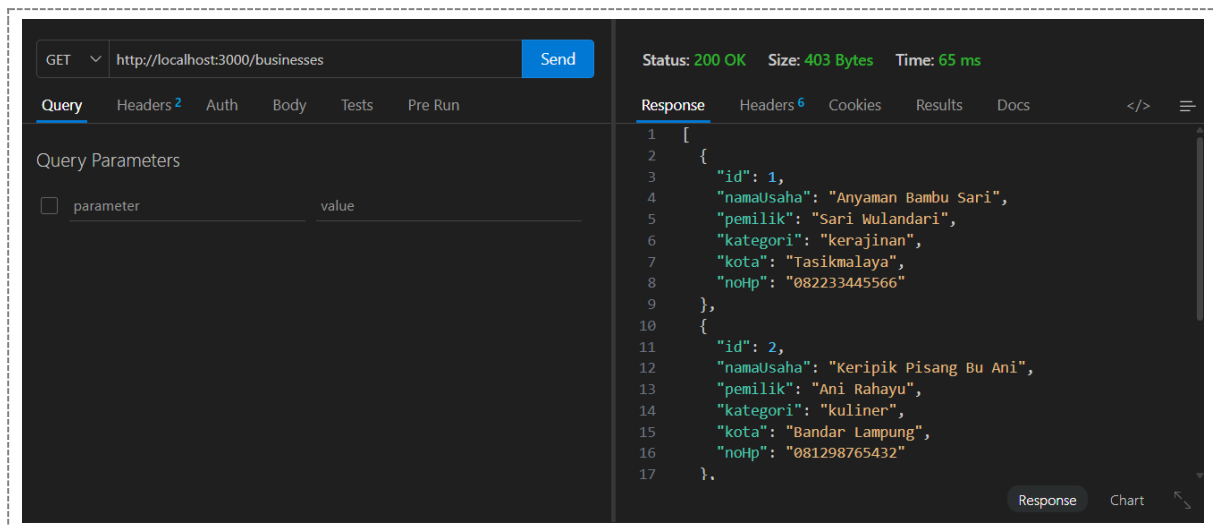
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /businesses)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /businesses
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; larik JSON berisi 3 data usaha (id 1, 2, dan 3).
Lulus?	☐ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

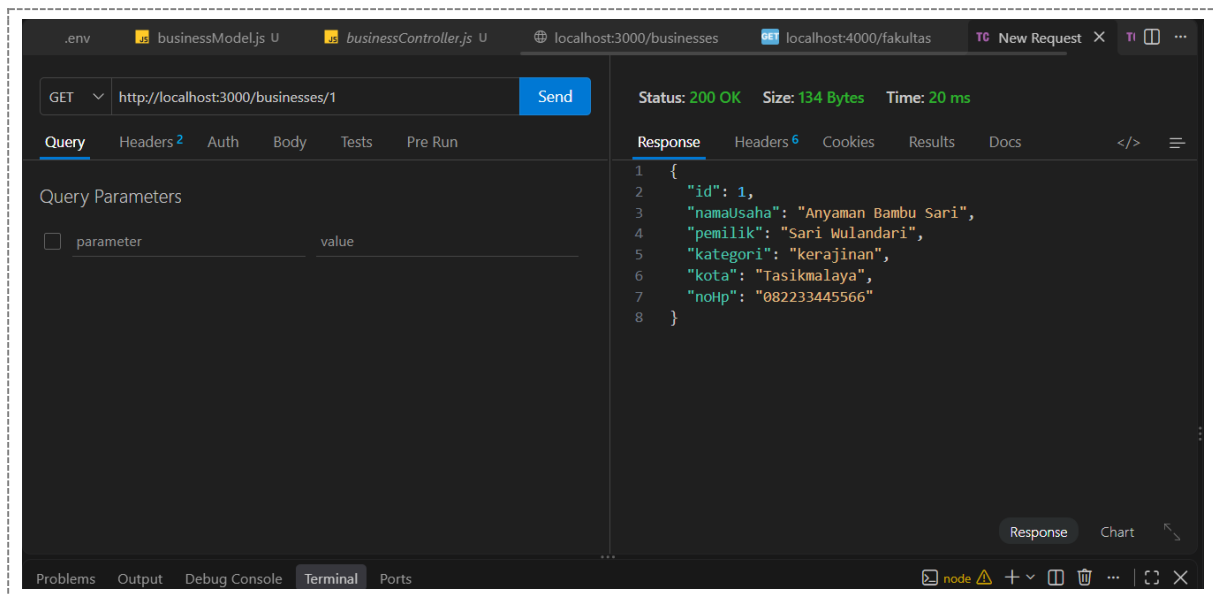
Penjelasan:

Status 200 OK berarti permintaan berhasil. Endpoint ini mengembalikan seluruh data usaha sebagai larik JSON, dan jika ditambah `?kategori=kuliner` hanya usaha berkategori itu yang dikembalikan. Alurnya: `app.use("/businesses", businessRoutes)` meneruskan request ke `router.get("/")` di `businessRoutes.js`, yang memanggil `getAllBusinesses` di controller. Controller mengambil `req.query.kategori` lalu memanggil `businessModel.getAll(kategori)` yang mengembalikan datanya.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /businesses/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; objek JSON usaha id 1 (Anyaman Bambu Sari).
Lulus?	[] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

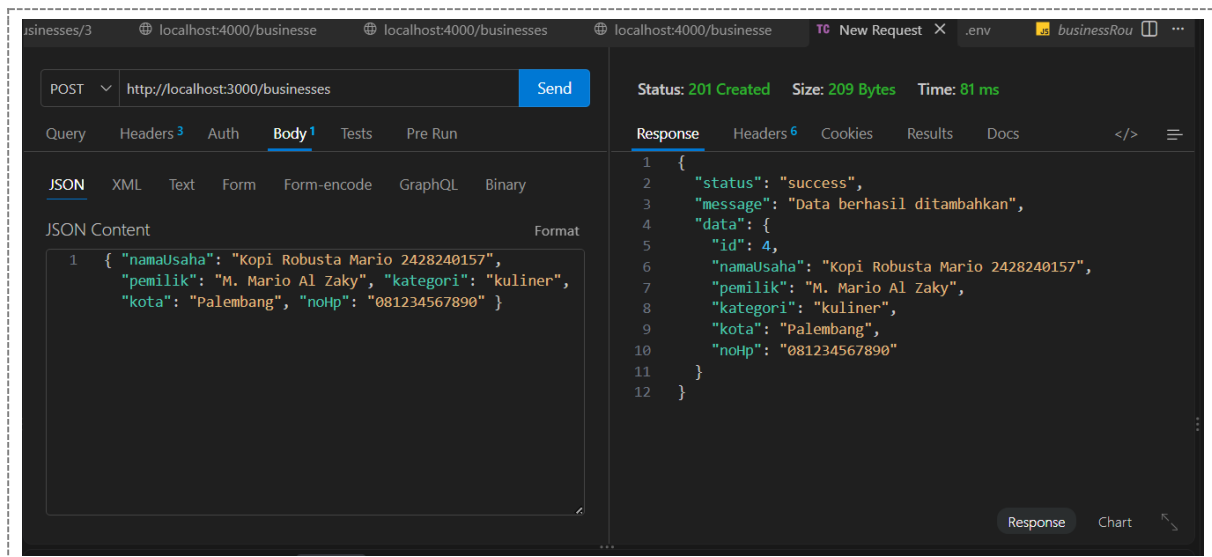
Penjelasan:

ID diambil dari alamat lewat `req.params.id`. Nilainya selalu berupa string ("1"), sedangkan id pada data bertipe angka dan pencarian memakai perbandingan ketat `===`, sehingga harus diubah dengan `parseInt` agar cocok. `Route router.get("/:id")` memanggil `getBusinessById`, yang memanggil `businessModel.getByid(id)` dan mengirim objek usaha tersebut dengan status 200.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /businesses
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaUsaha": "Kopi Robusta Mario 2428240157", "pemilik": "M. Mario Al Zaky", "kategori": "kuliner", "kota": "Palembang", "noHp": "081234567890" }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; {"status": "success", "message": "Data berhasil ditambahkan", "data": { id: 4, ... }}
Lulus?	[] Ya [] Tidak



Gambar 6. Hasil POST data baru

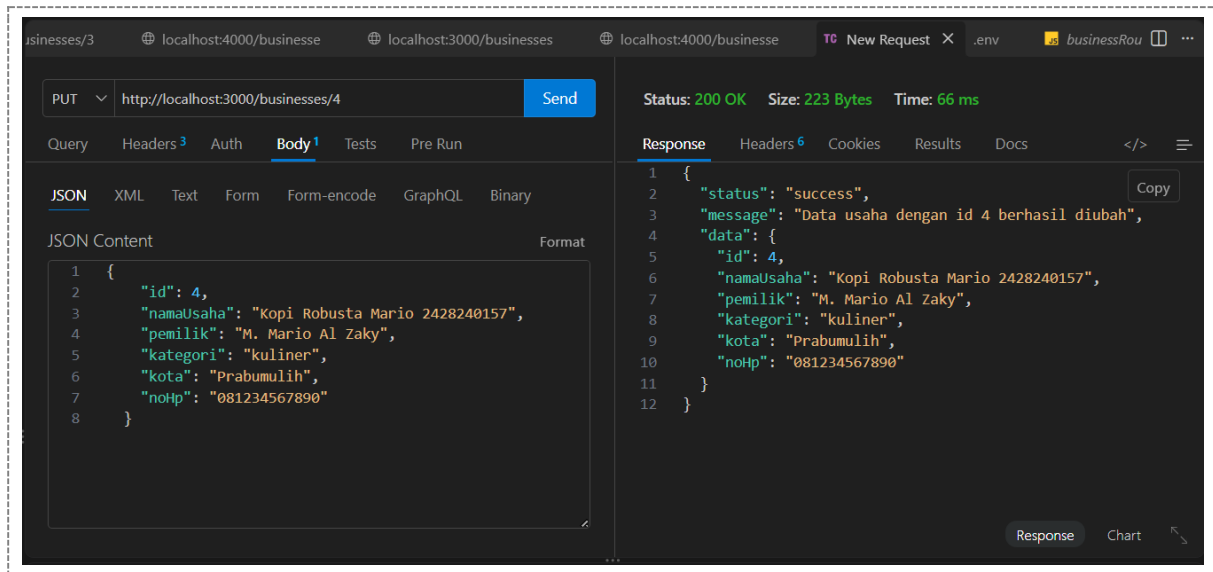
Penjelasan:

Header x-api-key diperiksa oleh middleware cekApiKey yang dipasang pada route POST, dengan membandingkannya dengan API_KEY di file .env. express.json() di app.js mengubah body JSON menjadi objek req.body sehingga createBusiness dapat membaca namaUsaha, pemilik, kategori, dan kota. Statusnya 201 Created, bukan 200, karena permintaan ini membuat resource baru, yaitu usaha dengan id baru dari nextId; 200 hanya berarti sukses secara umum.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /businesses/4
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "kota": "Prabumulih" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; { "status": "success", "message": "Data usaha dengan id 4 berhasil diubah", "data": { ... kota: "Prabumulih" } }
Lulus?	[] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

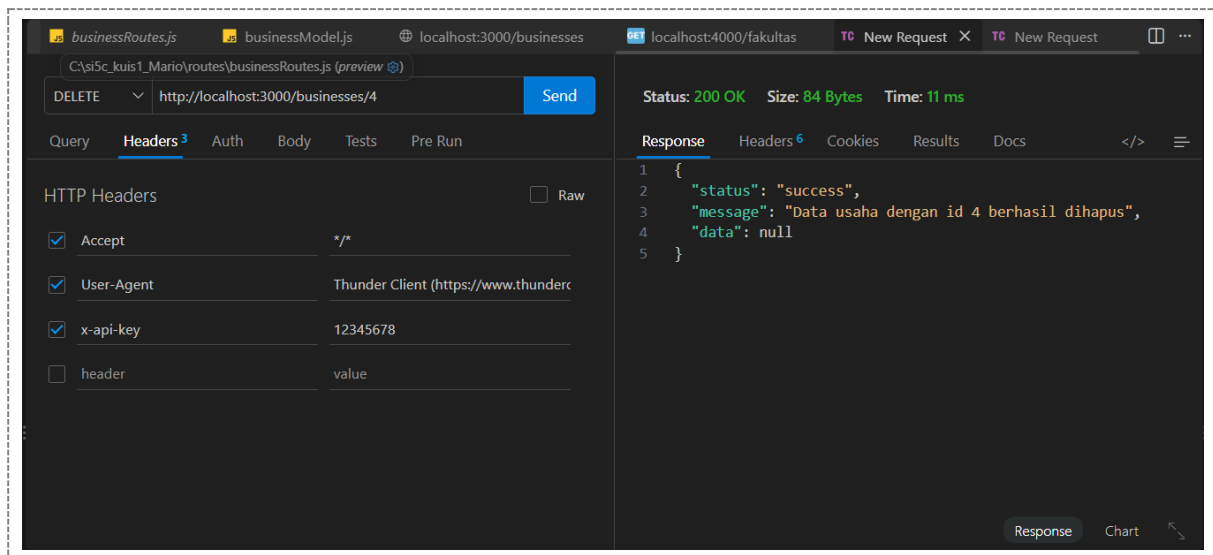
Penjelasan:

Hanya field kota yang berubah, dari "Palembang" menjadi "Prabumulih". Field id, namaUsaha, pemilik, kategori, dan noHp tetap karena model menggabungkan data lama dengan body baru lewat { ...businesses[index], ...data, id }. Perubahan dilakukan oleh layer Model (businessModel.update); controller hanya meneruskan req.body dan menyusun respons.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /businesses/4
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; {"status":"success","message":"Data usaha dengan id 4 berhasil dihapus"}
Status hasil uji	204 No Content; tanpa isi respons.
Lulus?	[] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content berarti permintaan berhasil dan data sudah dihapus, tetapi server sengaja tidak mengirim isi respons karena tidak ada lagi data yang perlu ditampilkan. Controller memanggil `businessModel.remove(id)` yang menghapus data dengan `splice`, lalu membalas lewat `res.status(204).send()`.

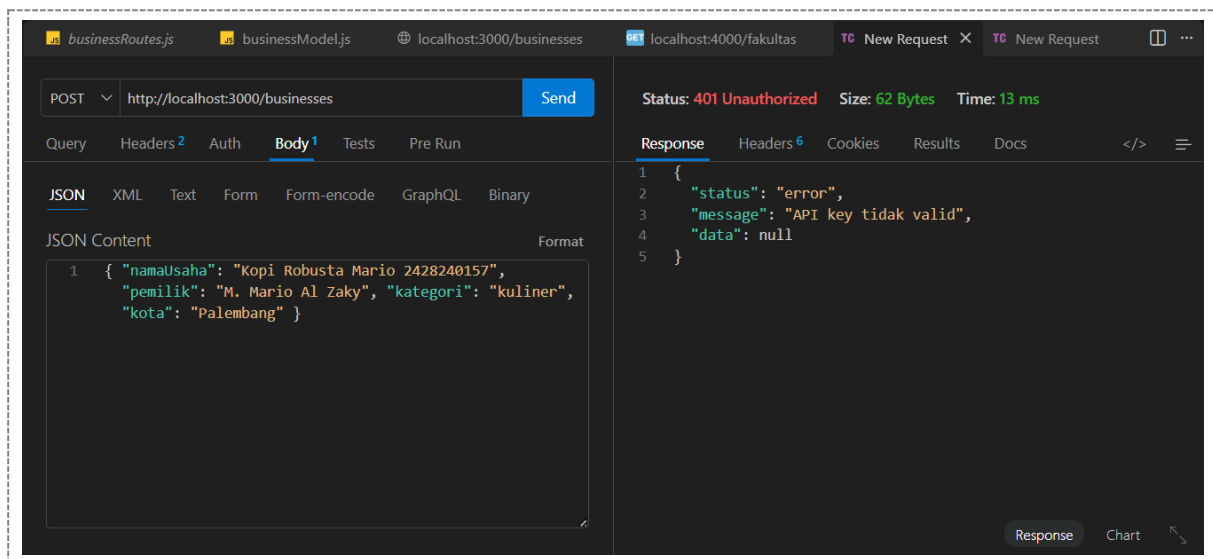
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /businesses (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "namaUsaha": "Kopi Robusta Mario 2428240157", "pemilik": "M. Mario Al Zaky", "kategori": "kuliner", "kota": "Palembang" }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; {"status": "error", "message": "API key tidak valid", "data": null}
Lulus?	☐ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

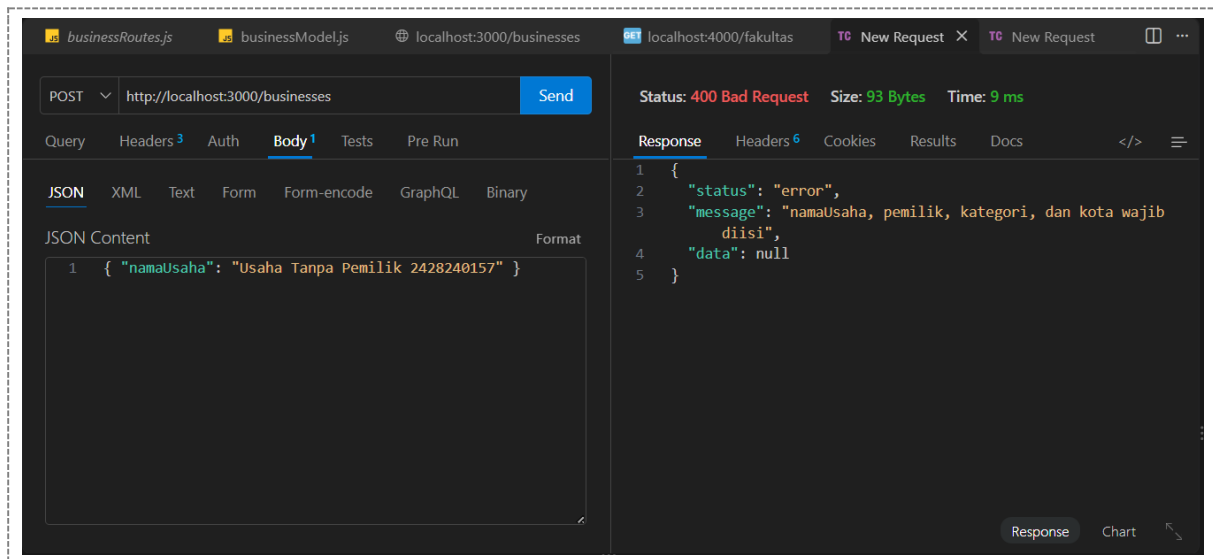
Penjelasan:

Yang menolak adalah middleware `cekApiKey`, yang memanggil `next(errorHttp(401, "API key tidak valid"))` karena header `x-api-key` tidak ada. Karena `cekApiKey` berada di rantai sebelum `createBusiness` dan tidak memanggil `next()` biasa, Express langsung melompat ke `errorHandler`. Akibatnya controller tidak sempat dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /businesses
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaUsaha": "Usaha Tanpa Pemilik 2428240157" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; {"status":"error","message":"namaUsaha, pemilik, kategori, dan kota wajib diisi","data":null}
Lulus?	☐ Ya ☐ Tidak



Gambar 10. POST dengan body tidak lengkap

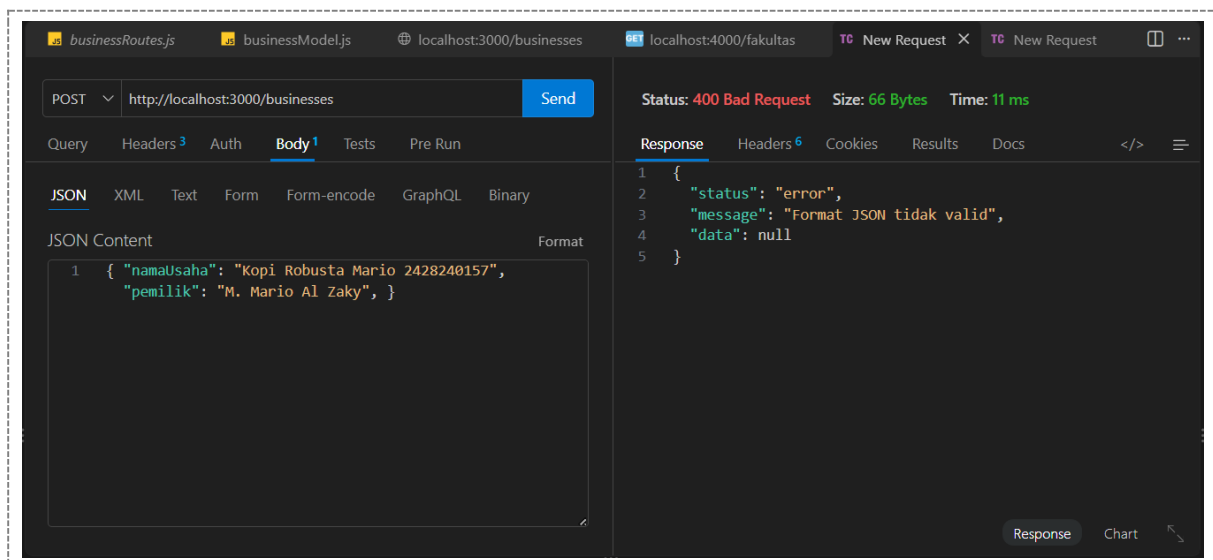
Penjelasan:

Validasi ditulis di layer Controller, pada fungsi `createBusiness`, lewat pengecekan `if (!namaUsaha || !pemilik || !kategori || !kota)`. Kode 400 Bad Request dipakai karena kesalahan ada pada data yang dikirim client (field wajib tidak lengkap), bukan pada server.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /businesses
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "namaUsaha": "Kopi Robusta Mario 2428240157", "pemilik": "M. Mario Al Zaky", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; {"status":"error","message":"Format JSON tidak valid","data":null}
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

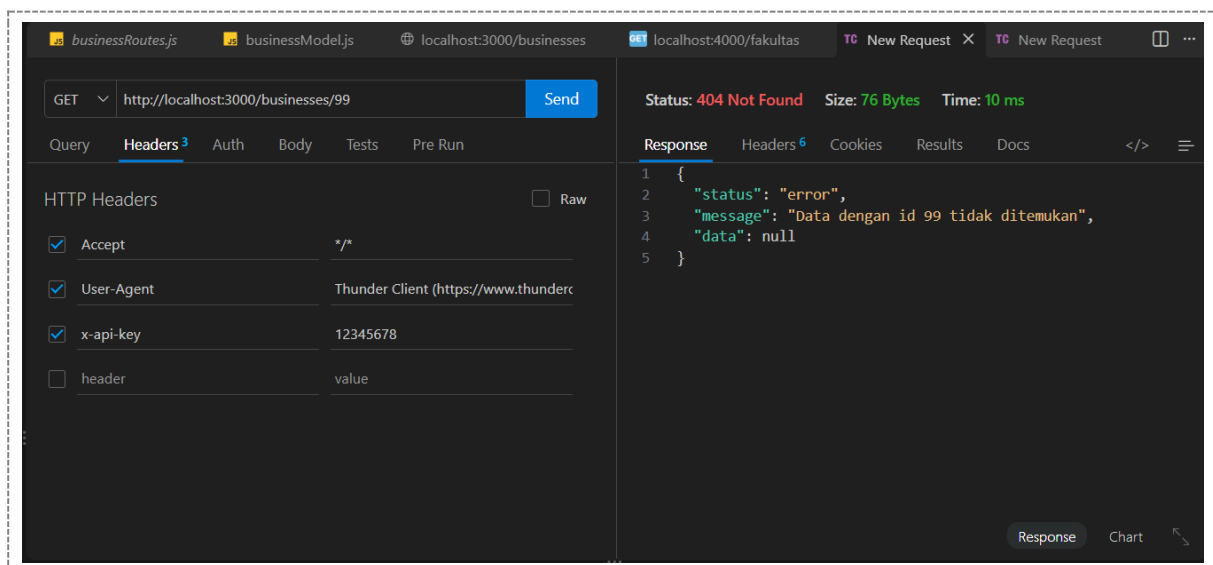
Penjelasan:

JSON rusak dideteksi oleh `express.json()` saat mem-parsing body, sebelum controller dijalankan; ia melempar error bertipe `entity.parse.failed`. `errorHandler` memeriksa `err.type` tersebut, lalu membalas 400 dengan JSON `{ status: "error", message: "Format JSON tidak valid", data: null }` sehingga hasilnya rapi dan bukan halaman HTML bawaan Express.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /businesses/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; {"status": "error", "message": "Data dengan id 99 tidak ditemukan", "data": null}
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request masuk ke router.get("/:id") lalu getBusinessById. businessModel.getById(99) mengembalikan undefined, sehingga controller memanggil next(errorHttp(404, "Data dengan id 99 tidak ditemukan")). Pesan itu dikirim oleh errorHandler di middlewares/errorHandler.js, yang membaca err.status dan err.message lalu membalas dengan JSON.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/Phumax12/si5c_kuis1_Mario
Pesan commit terakhir	.
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route hanya memetakan alamat ke fungsi, contohnya `router.post("/", cekApiKey, businessController.createBusiness)` di `businessRoutes.js`. Controller memproses request dan menyusun respons, contohnya `res.status(201).json({ ... })` di `createBusiness`. Model hanya mengelola data tanpa mengenal req atau res, contohnya `businesses.push(baru)` di `businessModel.create`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Dengan error handler terpusat, format dan pesan error cukup ditulis sekali di `errorHandler.js` sehingga semua endpoint membalas dengan struktur JSON yang sama, dan perubahan cukup dilakukan di satu tempat. Buktinya, N01 (401), N03 (400), dan N04 (404) berformat sama, `{ status, message, data }`, padahal berasal dari tempat berbeda: `cekApiKey`, `express.json()`, dan controller. JSON rusak pada N03 juga tertangani rapi tanpa kode tambahan di route. Jika `res.status(404)` ditulis di setiap route, formatnya mudah berbeda-beda dan error dari middleware seperti JSON rusak tidak ikut tertangani.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100